Part2–第一部分–附加一

1. 看论文

简单的看了一下三篇论文：

1.1 PRN 和 PReNet（CVPR 2019）

1.1.1 大概理解

链接的论文给了两个模型，一个PRN ，一个PReNet

这个论文作者说：

> Along with the deraining performance improvement of deep networks, their structures and learning become more and more complicated and diverse, making it difficult to analyze the contribution of various network modules when developing new deraining networks. To handle this issue, this paper provides a better and simpler baseline deraining network by considering network architecture, input and output, and loss functions

> 随着深度网络去雨性能的提升，其结构和学习变得越来越复杂多样，导致在开发新的去雨网络时难以分析各种网络模块的贡献。为解决这一问题，本文从网络架构、输入输出和损失函数三个方面出发，提供了一个更好且更简单的基线去雨网络

作者想解决：**给领域一个可分析的简单基线**

os：这确实很重要，提供对比基线

核心思路：

把一个很浅的 ResNet 反复用多次（多阶段）

PRN：

```
阶段 1:  y0 = B（有雨图）
          x1 = ResNet( [y0, B] )        ← 输入 6 通道
          y1 = y0 - x1                    ← 去雨
阶段 2:  x2 = ResNet( [y1, B] )
          y2 = y1 - x2
...
阶段 6:  y6 = y5 - x6                     ← 最终输出
```

Fence 1

PReNet：

PReNet = PRN + ConvLSTM

ConvLSTM层引入一个循环状态 $\mathbf{s}^t$

将每个阶段的输出与循环状态 $\mathbf{s}^t$ 一同输入一个卷积层，再输入下一阶段（具体数学原理还是太复杂了）

1.1.2 启示：

1. **不一定要发明新的计算指标公式，新的操作函数**，复用浅网络其实有时也能取得有效的结果
2. **多阶段要防信息丢失**，如同残差网络中，将head处理后的张量加回去，即每阶段拼原始输入
3. 参数不是越大越好，合适才是真理，先进的方法不一定效果好，正如论文原文：

4.3 真实雨天图像上的评估

利用图 9 中的两幅真实雨天图像，我们将 PReNet 与两种最先进的深度方法，即 JORDER [30] 和 DDN [6]，进行了比较。可以看出，PReNet 和 JORDER 在去除雨痕方面优于 DDN。对于第一幅图像，DDN 的结果中仍可见雨痕，而 PReNet 和 JORDER 能够生成令人满意的去雨结果。对于第二幅图像，DDN 和 JORDER 的结果中或多或少存在雨痕，而 PReNet 的结果更为清晰。

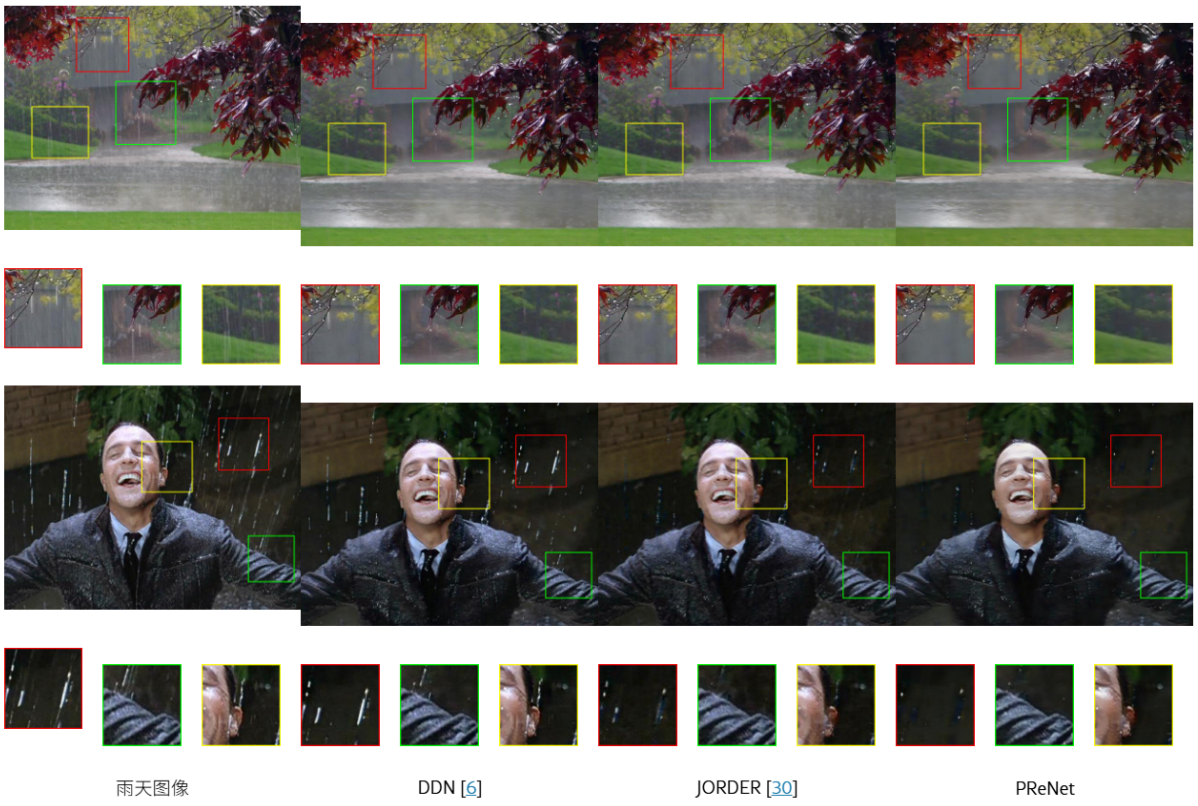


Figure 1

1.2 SPANet (CVPR 2019)

1.2.1 大概理解

雨丝是有方向的长条，所以用"沿 4 个方向扫"的注意力来捕捉它

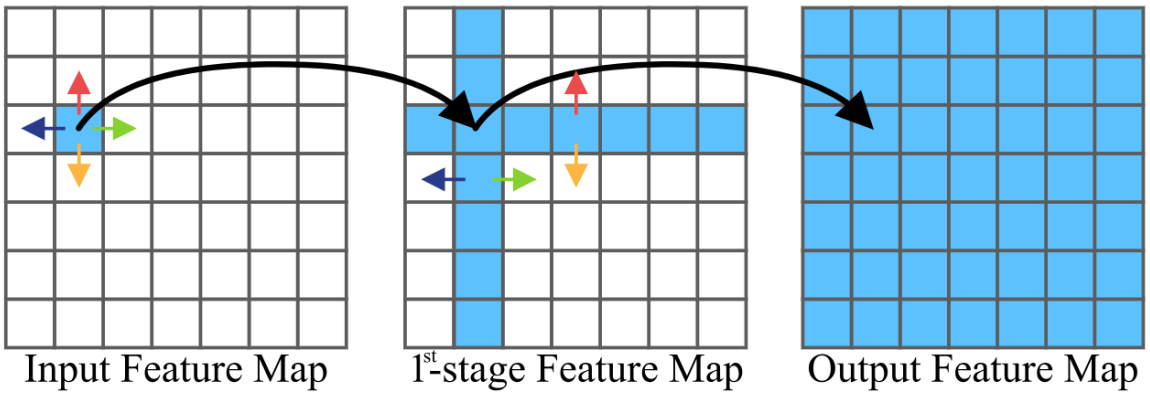


Figure 2

1.2.2 启示

1. 论文在开篇指出两个问题：

> **缺乏真实训练数据。** 由于无法获得真实的雨/清晰图像对，现有的去雨方法通常依赖合成数据集来训练模型。它们通常从一张清晰图像开始，并在其上添加合成雨，以构成雨/清晰图像对。尽管已有一些工作研究雨的物理特性，*例如*，雨的方向 [40] 和雨密度 [42]，但其数据集仍缺乏对广泛真实世界雨痕进行建模的能力。例如，通常很难像 [42] 中那样将雨密度分类为三个等级（即小雨、中雨和大雨）之一，而任何误分类都必然会影响去雨性能。为模拟全局雨效应，一些方法采用 Adobe Photoshop 中的非线性“滤色混合模式”，或额外在合成雨图像上叠加雾霾。然而，这些全局设置只能用于某些类型的雨，或者背景可能被变暗，从而丢失细节。
>
> **缺乏真实基准。** 目前，研究人员主要依赖通过视觉比较对真实雨图像上的去雨性能进行定性评估。Fan 等 [45] 还使用目标检测任务来辅助评估去雨性能。尽管如此，仍亟需一个高质量的真实去雨基准，以对去雨方法进行定量评估。
>
> 本文从两个方面解决单图像去雨问题，如图 1 所总结。首先，我们基于两点观察来解决真实训练/评估数据集缺乏的问题：(1) 由于随机雨滴以高速下落，它们不太可能始终覆盖同一像素 [13, 44]；(2) 在一段图像序列中，被雨覆盖的像素强度会在真实背景辐射亮度之上波动。这两点观察意味着，我们可以从一段雨图像序列中生成一幅干净图像，其中干净图像的各个像素可能来自该序列的不同图像。因此，我们提出一种半自动方法，将雨的时间特性与人工监督相结合，以构建大规模真实雨数据集。我们表明，该方法能够显著提升最先进去雨器在真实世界雨图像上的性能。
>
> 其次，我们观察到真实雨痕在单幅图像内可能呈现出高度多样的外观属性（*例如*，雨的形状和方向），这对现有的去雨方法构成了挑战，因为它们缺乏准确识别真实雨痕的能力。为解决这一局限，我们采用了一种空间注意力网络（SPANet），该网络首先利用水平/垂直邻域信息来建模雨痕的物理属性，然后通过进一步考虑非局部上下文信息来去除雨痕。通过这种方式，可以以从局部到全局的两阶段方式学习用于去除雨痕的判别性特征。大量评估表明，所提出的网络相较于最先进的去雨方法表现更优。

Figure 3

合成雨的建模不足，和真实雨差很远，真实雨图和无雨图**无法同时拍摄**，导致评估不客观

为了解决这个问题，他们改进数据集，使用半自动构造方法

```
1. 对同一场景拍一段连续视频（雨在动、背景不动）
2. 多帧信息能分离静态背景和瞬态雨丝
3. 半自动 + 人工监督，生成一张"无雨"的干净图
```

Fence 2

即：

数据集可能比模型更重要（好的数据集很大程度决定模型的训练好坏）

2. 抓住雨的方向特征（**利用任务的先验结构**）

   雨丝有方向，就用方向性模块，比通用卷积更对口

1.3 MPRNet (CVPR 2021)

1.3.1 大概理解

使用**多阶段渐进修复**：

每个阶段用"编码器-解码器（抓全局）+ 高分辨率分支（保细节）"双路结构

同时引入逐像素自适应，决定图像不同地方要修多少的程度

1.3.2 启发

使用编码器-解码器 + 高分辨率分支这样的双路结构

同时使用多阶段的模型结构

2．方法总结

2.1 找文章

找文章最关键的是弄清楚要什么和手里有什么线索，掌握丰富的网站：

1．弄清楚自己要什么领域，与什么相关，什么层次的论文

一篇已知的论文还是**一个主题**

2．线索的丰富程度决定用哪个网站、用什么查

查一篇已知的论文有**DOI**，**标题**，**作者+年份+期刊** 很好查

考虑在谷歌学术或 Web of Science 精确匹配，或者检索作者，年份，期刊

如果是一个主题（比如去雨），去该主题所属领域权威的机构官方的期刊和官网对应板块查询，或者关键字搜索

3．引用检索：

论文结尾会有引用资料的链接，这往往是相关领域的文章

如果想找一个模型或者概念的开创文章，可以通过引用链一层一层回溯

2.2 处理一篇文章

可以尝试使用专门训练用来学术翻译的ai翻译服务，翻译全文，了解大概

当然全靠ai翻译是不可靠的（**ai也可能会出现翻译问题**）

要精读时可以对重要部分看原文，查阅学术词汇

面对不会的尝试查资料，询问专业人士，或者向ai求助

较新的文章出现新的术语与概念最好不要只发概念词，**把上下文也要一同发过去**

2.3 渐进式训练

渐进式训练是指在训练过程中不使用一套恒定不变的参数，而是动态调整参数的训练范式

在图像修复领域渐进式训练通常指的是渐进增大 patch 的训练过程

先在小 patch 上训，逐步换成大 patch

patch：从原图裁剪的尺寸，相当于先训练局部图像，再慢慢扩大，训练整张图

而出题人从始至终用的原图大小固定

这样做的好处：

1．小 patch 迭代快，梯度小，稳定，早期计算可以快速收敛，可以早期快速发现模型设计的缺陷和问题

2．间接扩大数据集，每轮从不同位置裁，同一张图能提供多个不同样本

劣势：

1．实现复杂，过程繁琐

2．需要的超参数要实时调整适应更大的patch

未来科研是否用：

渐进式训练是一种高效的方法，但是不能滥用，也不能原版照抄已有文章的渐进式训练的超参数设置，需要考虑自身的时间，精力，训练的目的，要求，结果的精度要求等等因素

3．自己的模型

下面是自己设计的模型：

主要在之前的dedrainnet中加了丐版的4方向累积注意力机制

但是这样模型参数量已经达到了2.41M，在epoch=200，训练图片只有1800张的情况下可能会过拟合

```
import torch
import torch.nn as nn
import torch.nn.functional as F

# 依旧残差块起手
class ResidualBlock(nn.Module):
        def __init__(self, channels=64):
            super().__init__()
            self.body = nn.Sequential(
                nn.Conv2d(channels, channels, kernel_size=3, stride=1, padding=1,bias=False),
                nn.InstanceNorm2d(channels),
                nn.ReLU(inplace=True),
                nn.Conv2d(channels, channels, kernel_size=3, stride=1, padding=1,bias=False),
                nn.InstanceNorm2d(channels),
            )

        def forward(self,x):
            y = self.body(x)
            return F.relu(x + y)


# 丐版的4方向累积注意力机制
class SPANet(nn.Module):
        def __init__(self,channels=64):
            super().__init__()
            self.conv1 = nn.Conv2d(channels * 4,channels,kernel_size=3,stride=1,padding=1)
            self.conv2 = nn.Conv2d(channels,1,kernel_size=3,stride=1,padding=1)
        def forward(self,x):
            right = torch.cumsum(x,dim=3)
            left = torch.cumsum(x.flip(3),dim=3).flip(3)
            down = torch.cumsum(x,dim=2)
            up = torch.cumsum(x.flip(2),dim=2).flip(2)
            full = self.conv1(torch.cat([right,left,down,up],dim=1))
            a = torch.sigmoid(self.conv2(full))
            return x * (a + 1)

class Mymodel(nn.Module):
        def __init__(self, raw_channels=3, inter_channels=64, num_blocks=8):
            super().__init__()
            # 初处理，把channels从3变成64
            self.head = nn.Conv2d(raw_channels,inter_channels, kernel_size=3, stride=1, padding=1)
            self.rest_full    = nn.Sequential(*[ResidualBlock(64) for _ in range(num_blocks)])
            self.rest_half    = nn.Sequential(*[ResidualBlock(64) for _ in range(num_blocks)])
            self.rest_quarter = nn.Sequential(*[ResidualBlock(64) for _ in range(num_blocks)])
            # 池化，造分支
            self.pool_half = nn.MaxPool2d(kernel_size=2,stride=2)
            self.pool_quarter = nn.MaxPool2d(kernel_size=4,stride=4)
            # 转置卷积，变回原图
            self.back_double =
nn.ConvTranspose2d(inter_channels,inter_channels,kernel_size=2,stride=2)
            self.back_four =
nn.ConvTranspose2d(inter_channels,inter_channels,kernel_size=4,stride=4)
            # 尾处理，把channels从64变成3
            self.tail = nn.Sequential(
                nn.Conv2d(inter_channels *
3,inter_channels,kernel_size=3,stride=1,padding=1,bias=False),
                nn.InstanceNorm2d(inter_channels),
                nn.ReLU(inplace=True),
                nn.Conv2d(inter_channels, raw_channels,kernel_size=3,stride=1,padding=1),
            )
            nn.init.zeros_(self.tail[-1].weight)
            nn.init.zeros_(self.tail[-1].bias)
            self.spa1 = SPANet()
```

```
            self.spa2 = SPANet()
            self.spa3 = SPANet()
        def forward(self,x):
            x_0 = self.head(x)

            h_1 = x_0
            b_1 = self.rest_full(h_1) + h_1
            x_1 = self.spa1(b_1)

            h_2 = self.pool_half(x_0)
            b_2 = self.back_double(self.rest_half(h_2) + h_2 )
            x_2 = self.spa2(b_2)

            h_3 = self.pool_quarter(x_0)
            b_3 = self.back_four(self.rest_quarter(h_3) + h_3 )
            x_3 = self.spa3(b_3)

            inter = torch.cat([x_1,x_2,x_3], dim= 1)
            rain = self.tail(inter)
            return x - rain
```

Fence 3

训练指标：

数据如下：



Figure 4

与之前的derainnet相比：

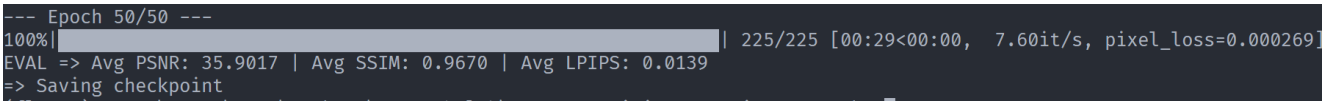


Figure 5

PSNR要差点，SSIM差不多，但是LPIPS要好一点（其实差不多）

测试指标：

mymodel：

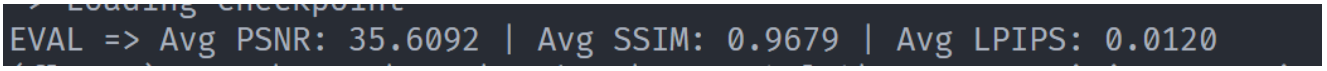


Figure 6

derainnet：



Figure 7

其实mymodel更好些

4．简单的计算模型参数的代码

计算模型参数的代码：

```
import os
import torch
from torch.utils.data import DataLoader
from models.mymodel import Mymodel
from dataset import DerainDataset

def main():
        device = torch.device("cpu")
        model = Mymodel().to(device).eval()
```

```
        root_dir = "./data/Rain200L"
        dataset = DerainDataset(root_dir,is_train=False)
        rainy, _, _ = next(iter(DataLoader(dataset,batch_size=1)))
        rainy = rainy.to(device)

        checkpoint_path = "savemodel_onnx"
        mymodel_onnx_path = os.path.join(checkpoint_path,"mymodel.onnx")
        os.makedirs(checkpoint_path,exist_ok=True)

        n = sum(p.numel() for p in model.parameters())
        print(f"模型参数量大小为{n / 1e6 :.3f}M")
        torch.onnx.export(model,rainy,mymodel_onnx_path,export_params=True,dynamo=False,
                          opset_version=17,input_names=["rainy"],output_names=["derain"],
                          dynamic_axes={
                              "rainy":{0:"batch",2:"height",3:"width"},
                              "derain":{0:"batch",2:"height",3:"width"}
                          },
                          )
        print(f"已把mymodel导出至{mymodel_onnx_path},文件大小为{os.path.getsize(mymodel_onnx_path) / (1024 ** 2):.2f} MB")

if __name__ == "__main__":
        main()

```

Fence 4

输出截图:



Figure 8

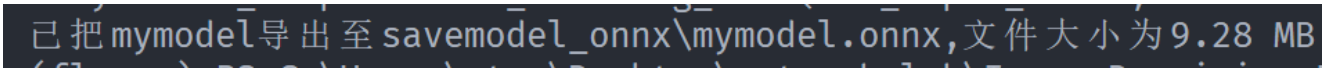
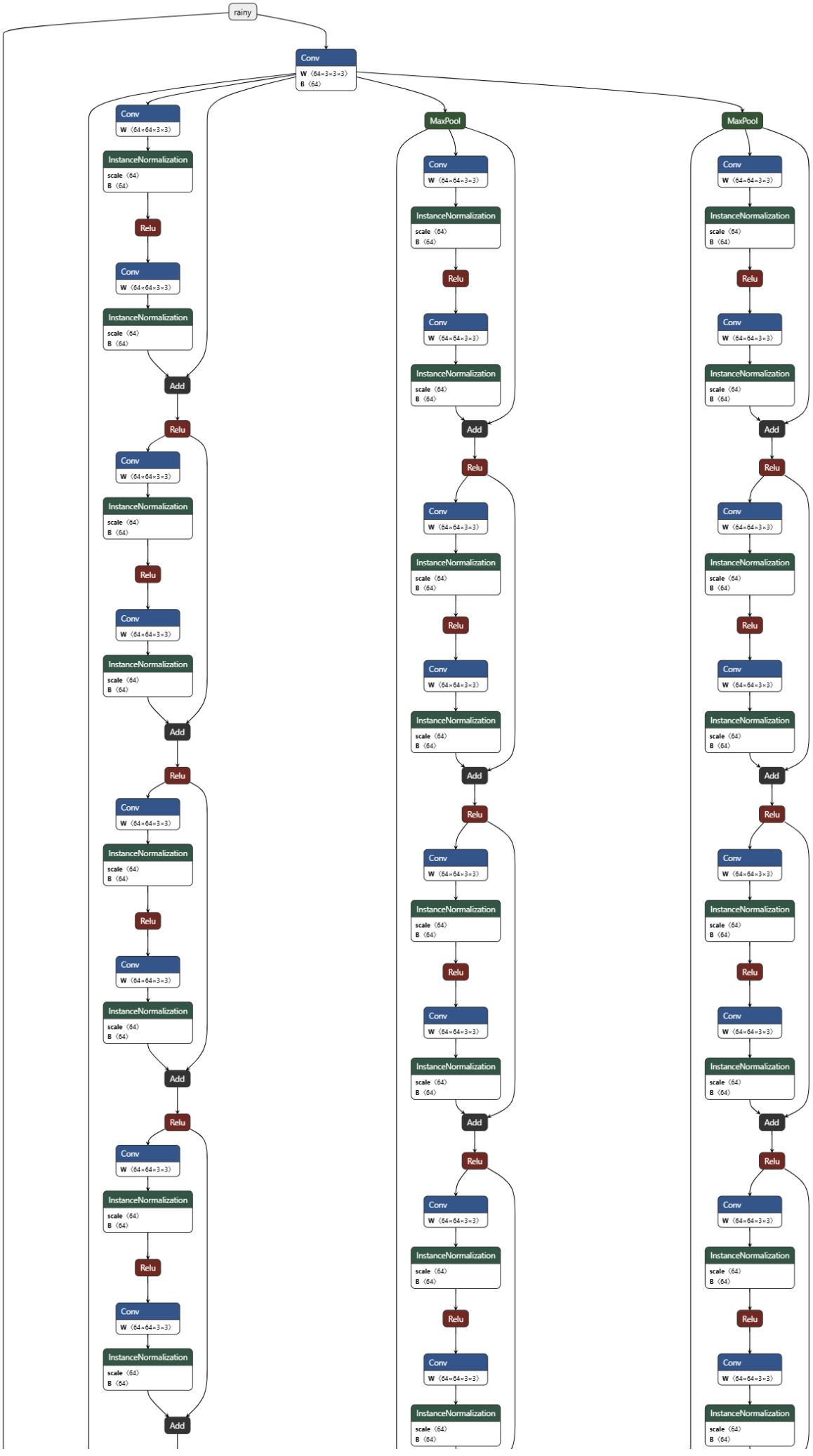
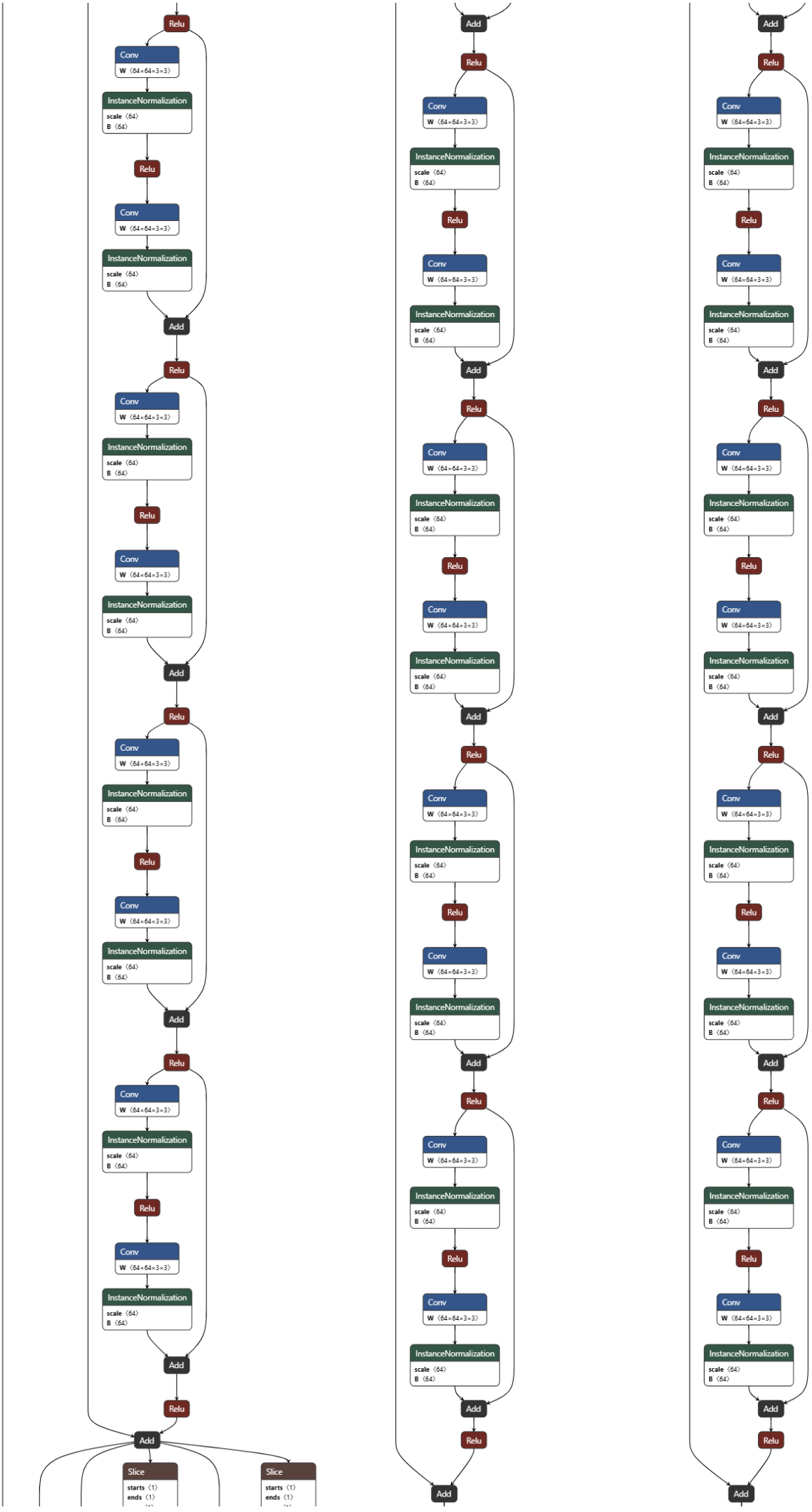


Figure 9

拖入网站:


rainy
Conv
W 〈64×3×3×3〉
B 〈64〉
MaxPool
MaxPool
Conv
W 〈64×64×3×3〉
InstanceNormalization
scale 〈64〉
B 〈64〉
Relu
Add


Relu
Conv
W ⟨64×64×3×3⟩
InstanceNormalization
scale ⟨64⟩
B ⟨64⟩
Relu
Conv
W ⟨64×64×3×3⟩
InstanceNormalization
scale ⟨64⟩
B ⟨64⟩
Add
Relu
Conv
W ⟨64×64×3×3⟩
InstanceNormalization
scale ⟨64⟩
B ⟨64⟩
Relu
Conv
W ⟨64×64×3×3⟩
InstanceNormalization
scale ⟨64⟩
B ⟨64⟩
Add
Relu
Conv
W ⟨64×64×3×3⟩
InstanceNormalization
scale ⟨64⟩
B ⟨64⟩
Relu
Conv
W ⟨64×64×3×3⟩
InstanceNormalization
scale ⟨64⟩
B ⟨64⟩
Add
Relu
Conv
W ⟨64×64×3×3⟩
InstanceNormalization
scale ⟨64⟩
B ⟨64⟩
Relu
Conv
W ⟨64×64×3×3⟩
InstanceNormalization
scale ⟨64⟩
B ⟨64⟩
Add
Relu
Conv
W ⟨64×64×3×3⟩
InstanceNormalization
scale ⟨64⟩
B ⟨64⟩
Relu
Conv
W ⟨64×64×3×3⟩
InstanceNormalization
scale ⟨64⟩
B ⟨64⟩
Add
Relu
Add
Slice
starts ⟨1⟩
ends ⟨1⟩
Slice
starts ⟨1⟩
ends ⟨1⟩

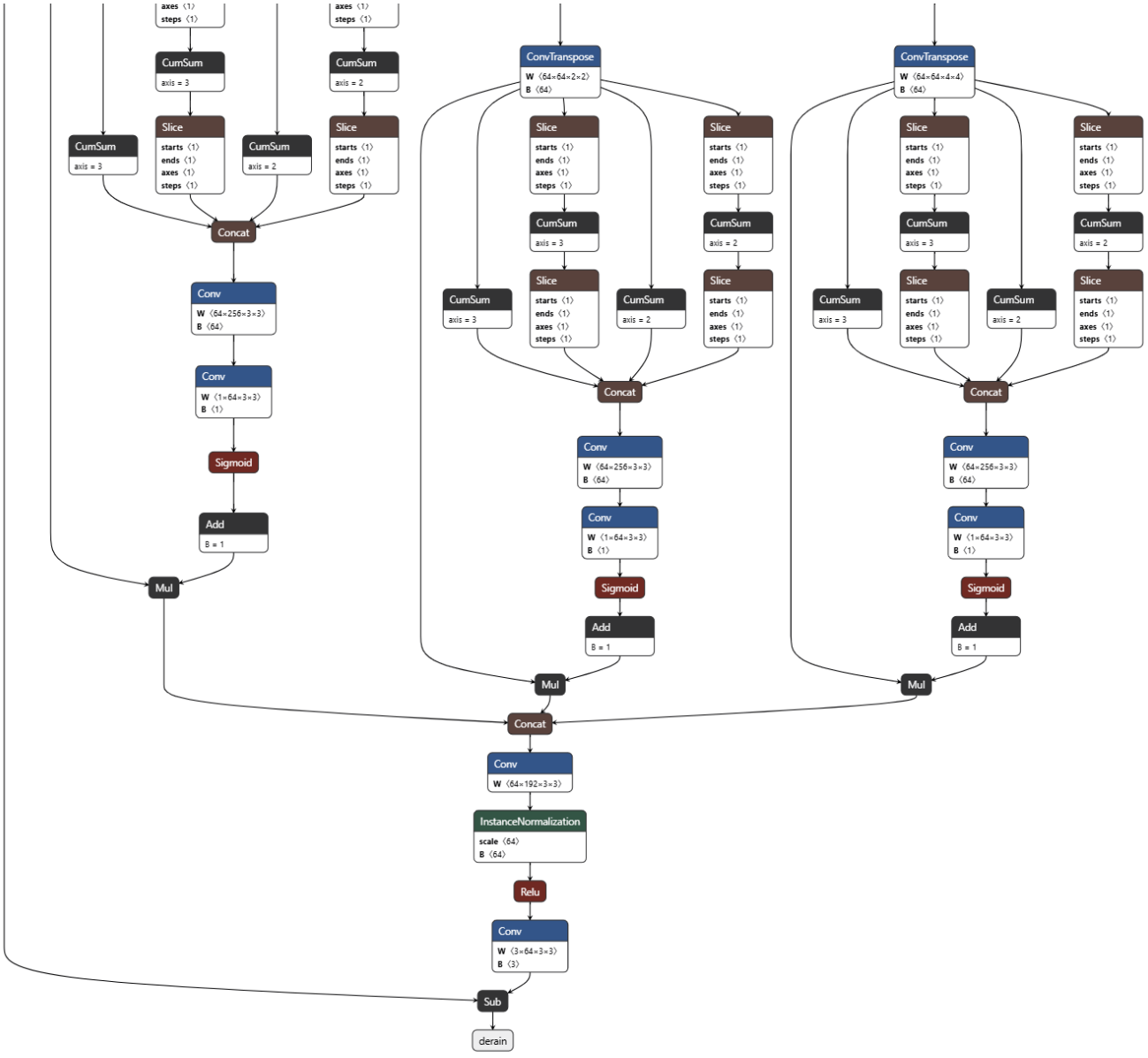


Figure 10

5．为什么使用文中同样的架构或模块来优化模型反而不会提高，甚至降低点数?

适合的才是最好的

这个任务的训练集只有1800张图，不同模型架构训练集的“舒适区”不同（小了过拟合，多了欠拟合，时间也长）

除此之外，训练集的质量也很重要，正如SPANet论文中提到，合成雨的建模不足，和真实雨差很远，图像建模不好也会很大程度影响训练效果

而且不同模型所需的优化器、调度器也要对应进行合适的选择，采取渐进式和不采取训练效果也会不同

简而言之，不能只生搬硬套论文的模型结构，要提炼思想，基于自己的任务进行优化